PROPOSED SOLUTION

Implement Client Script & UI Policy (Incident)

1. Proposed Solution

To enforce data integrity on Incident records at the point of entry, the solution combines two complementary ServiceNow client-side mechanisms:

- **UI Policies & UI Policy Actions** — Declarative rules that dynamically control field mandatory status, visibility, and read-only state based on conditions (Priority, State, Category). Preferred for straightforward conditional behavior and ease of maintenance.
- **Client Scripts** — JavaScript (onLoad, onChange, onSubmit) for advanced logic: auto-population of related fields, clearing of dependent values, cross-field validation, and blocking form submission when business rules are not met.

Together they provide a layered approach: UI Policies handle the majority of conditional UI behavior, while Client Scripts cover richer validation and dynamic population that cannot be expressed declaratively. This ensures Incidents are created with complete and valid information before they are submitted.

2. Detailed Solution Design

2.1 UI Policies & UI Policy Actions

The following UI Policies enforce key business rules on the Incident form. Each policy has corresponding UI Policy Actions; Reverse-if-false is enabled where appropriate.

UI Policy	Condition	Actions
Assignment Group mandatory for high priority	Priority is 1 – Critical or 2 – High	Assignment group → Mandatory = true
Show Business Impact for P1/P2	Priority is 1 or 2	Business impact fields → Visible & Mandatory = true
Subcategory mandatory when Category set	Category is not empty	Subcategory → Mandatory = true
Read-only after resolution	State is Resolved or Closed	Short description, Description, Category, Priority → Read-only = true
Caller mandatory on create	Always (new records)	Caller → Mandatory = true

Table 1: Proposed UI Policies

2.2 Client Scripts

Client Scripts complement UI Policies with imperative logic for auto-population, dependent field handling, and save-time validation:

Script / Type	Trigger	Purpose & Key Logic
Auto-set Urgency from Priority	onChange – Priority	Maps Priority → Urgency to keep the priority matrix consistent.
Validate Description length	onSubmit	Blocks save if Description is empty or below minimum length; shows clear error message.
P1 Assignment Group guard	onSubmit	Defense-in-depth: ensures Assignment group is populated for Critical incidents.
Clear dependent fields	onChange – Category	Clears Subcategory when Category changes so invalid combinations cannot remain.
Default Category on load	onLoad (new records)	Optionally pre-fills Category based on user role/department to speed data entry.

Table 2: Proposed Client Scripts

2.3 Form Validation Strategy

Validation is applied at two levels for robustness: **UI Policy (Mandatory)** provides immediate visual indication (red asterisk) and blocks save for simple required-field rules; **Client Script (onSubmit)** performs richer checks (length, cross-field dependencies) and displays custom messages via g_form APIs, returning false to abort the save. This layered approach ensures incomplete records cannot be submitted.

3. Expected Benefits & Outcomes

Implementing the proposed UI Policies and Client Scripts delivers the following benefits:

Benefit	Description
Improved Data Quality	Incidents are created with complete, consistent information. Mandatory fields and validation rules significantly reduce incomplete or incorrect submissions, lowering the need for follow-up and rework by support teams.
Better SLA Compliance	Critical fields required for proper prioritization and assignment (e.g., Assignment Group for P1/P2) are enforced at entry. This supports accurate prioritization, correct routing, and reliable SLA tracking and reporting.
Reduced User Error	Conditional mandatory status, field visibility, and read-only controls guide users in real time. Dependent fields are cleared automatically, preventing invalid combinations and common data-entry mistakes.
Faster Triage & Routing	Auto-population of related values (e.g., Urgency from Priority) and enforced Assignment Group for high-priority tickets accelerate routing to the correct resolver groups, shortening mean time to assign.
Maintainable Configuration	Preference for declarative UI Policies over pure Client Scripts where possible keeps the solution understandable and maintainable by process owners, reducing long-term support overhead.
Audit & Reporting Confidence	Higher-quality, consistent Incident data improves the reliability of operational dashboards, management reports, and compliance evidence derived from the Incident table.

Table 3: Expected Benefits

By combining UI Policies, UI Policy Actions, and Client Scripts on the Incident form, the organization enforces data integrity at the point of entry—directly addressing incomplete or inconsistent Incident data, supporting accurate triage and SLA management, and demonstrating practical application of Incident Management, UI Policy, Client Scripts, and Form Validation skills.